

HDOC Day - 41

Name - Aaryan Pratap Singh

Sap- 590032842

Batch - 15

1. Algorithm

1. Start.
2. Declare two character arrays `paragraph1` and `paragraph2`.
3. Call `readParagraph ( )` twice to read both paragraphs.
4. Repeatedly display the menu:
 - Find longest and shortest words.
 - Remove repeated consecutive spaces.
 - Convert paragraphs to title case.
 - Find frequency of every distinct word.
 - Exit.
5. Read the user's choice.
6. Use `switch` to perform the selected operation.
7. For **Choice 1**:
 - Call `findLongestAndShortestWords ( )` for both paragraphs.
 - Tokenize a copy of the paragraph using `strtok ( )`.
 - Compare word lengths using `strlen ( )`.
 - Store the longest and shortest words using `strcpy ( )`.
8. For **Choice 2**:
 - Call `removeRepeatedSpaces ( )` for both paragraphs.
 - Scan each paragraph character by character.
 - If whitespace occurs repeatedly, store only one space.
9. For **Choice 3**:
 - Call `convertToTitleCase ( )` for both paragraphs.
 - Convert the first alphabetic character of each word to uppercase.
 - Convert the remaining alphabetic characters to lowercase.
10. For **Choice 4**:
 - Call `findWordFrequencies ( )` for each paragraph.
 - Convert words to lowercase so comparison is case-insensitive.
 - Count each distinct word and display its frequency.
11. For **Choice 5**, terminate the program.
12. For any other choice, display an error message.
13. Stop.

2. C Program

```
#include <stdio.h>
#include <string.h>
```

Aaryan Pratap Singh

590032842

B-15

```

#include <ctype.h>

#define MAX_SIZE 1000
#define MAX_WORDS 200
#define MAX_WORD_LENGTH 100

void readParagraph(char paragraph[], size_t size);
void findLongestAndShortestWords(const char paragraph[],
                                char longestWord[],
                                char shortestWord[]);
void removeRepeatedSpaces(const char paragraph[],
                           char modifiedParagraph[]);
void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[]);
void findWordFrequencies(const char paragraph[]);

int main()
{
    char paragraph1[MAX_SIZE], paragraph2[MAX_SIZE];
    char longest[MAX_WORD_LENGTH], shortest[MAX_WORD_LENGTH];
    char modified[MAX_SIZE], titleCase[MAX_SIZE];
    int choice;

    printf("Enter paragraph 1:\n");
    readParagraph(paragraph1, MAX_SIZE);

    printf("\nEnter paragraph 2:\n");
    readParagraph(paragraph2, MAX_SIZE);

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Find longest and shortest words\n");
        printf("2. Remove repeated consecutive spaces\n");
        printf("3. Convert paragraphs to title case\n");
        printf("4. Find frequency of every distinct word\n");
        printf("5. Exit\n");
        printf("===== \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("\n--- Paragraph 1 ---\n");

```

```

        findLongestAndShortestWords(paragraph1,
                                     longest,
shortest);

    printf("Longest word  : %s\n", longest);
    printf("Shortest word : %s\n", shortest);

    printf("\n--- Paragraph 2 ---\n");

    findLongestAndShortestWords(paragraph2,
                                     longest,
shortest);

    printf("Longest word  : %s\n", longest);
    printf("Shortest word : %s\n", shortest);
    break;

case 2:
    printf("\n--- Paragraph 1 ---\n");

    removeRepeatedSpaces(paragraph1, modified);

    printf("Modified paragraph:\n%s\n",
modified);

    printf("\n--- Paragraph 2 ---\n");

    removeRepeatedSpaces(paragraph2, modified);

    printf("Modified paragraph:\n%s\n",
modified);

    break;

case 3:
    printf("\n--- Paragraph 1 ---\n");

    convertToTitleCase(paragraph1, titleCase);

    printf("Title case:\n%s\n", titleCase);

    printf("\n--- Paragraph 2 ---\n");

    convertToTitleCase(paragraph2, titleCase);

```

```

        printf("Title case:\n%s\n", titleCase);
        break;

    case 4:
        printf("\n--- Paragraph 1 ---\n");
        findWordFrequencies(paragraph1);

        printf("\n--- Paragraph 2 ---\n");
        findWordFrequencies(paragraph2);
        break;

    case 5:
        printf("\nExiting program...\n");
        break;

    default:
        printf("\nInvalid choice! Please enter 1 to
5.\n");
    }

    } while (choice != 5);

    return 0;
}

/* Function to read a paragraph */
void readParagraph(char paragraph[], size_t size)
{
    fgets(paragraph, size, stdin);

    /* Remove newline character */
    paragraph[strcspn(paragraph, "\n")] = '\0';
}

/* Function to find longest and shortest words */
void findLongestAndShortestWords(const char paragraph[],
                                char longestWord[],
                                char shortestWord[])
{
    char temp[MAX_SIZE];
    char *word;

    strcpy(temp, paragraph);

```

```

word = strtok(temp, " \t\n.,!?:\\"'()[ ]{}");

if (word == NULL)
{
    strcpy(longestWord, "");
    strcpy(shortestWord, "");
    return;
}

strcpy(longestWord, word);
strcpy(shortestWord, word);

while (word != NULL)
{
    if (strlen(word) > strlen(longestWord))
    {
        strcpy(longestWord, word);
    }

    if (strlen(word) < strlen(shortestWord))
    {
        strcpy(shortestWord, word);
    }

    word = strtok(NULL, " \t\n.,!?:\\"'()[ ]{}");
}
}

```

```

/* Function to remove repeated spaces */
void removeRepeatedSpaces(const char paragraph[],
                          char modifiedParagraph[])
{
    int i = 0;
    int j = 0;
    int previousSpace = 0;

    while (paragraph[i] != '\0')
    {
        if (isspace((unsigned char)paragraph[i]))
        {
            if (!previousSpace)
            {
                modifiedParagraph[j++] = ' ';
            }
        }
        else
        {
            modifiedParagraph[j++] = paragraph[i];
        }
        i++;
    }
    modifiedParagraph[j] = '\0';
}

```

```

        previousSpace = 1;
    }
}
else
{
    modifiedParagraph[j++] = paragraph[i];
    previousSpace = 0;
}

    i++;
}

modifiedParagraph[j] = '\0';
}

/* Function to convert paragraph to title case */
void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[])
{
    int i = 0;
    int newWord = 1;

    while (paragraph[i] != '\0')
    {
        if (isalpha((unsigned char)paragraph[i]))
        {
            if (newWord)
            {
                titleCaseParagraph[i] =
                    toupper((unsigned char)paragraph[i]);

                newWord = 0;
            }
            else
            {
                titleCaseParagraph[i] =
                    tolower((unsigned char)paragraph[i]);
            }
        }
        else
        {
            titleCaseParagraph[i] = paragraph[i];

            if (isspace((unsigned char)paragraph[i]))

```

```

        {
            newWord = 1;
        }
    }

    i++;
}

titleCaseParagraph[i] = '\0';
}

/* Function to find word frequencies */
void findWordFrequencies(const char paragraph[])
{
    char temp[MAX_SIZE];
    char words[MAX_WORDS][MAX_WORD_LENGTH];
    int frequency[MAX_WORDS] = {0};

    char *word;
    int wordCount = 0;
    int found;
    int i, j;

    strcpy(temp, paragraph);

    word = strtok(temp, " \\t\\n.,!?;:\\\"'()[\\]{}");

    while (word != NULL)
    {
        /* Convert word to lowercase */
        for (i = 0; word[i] != '\\0'; i++)
        {
            word[i] = tolower((unsigned char)word[i]);
        }

        found = -1;

        /* Search if the word already exists */
        for (i = 0; i < wordCount; i++)
        {
            if (strcmp(words[i], word) == 0)
            {
                found = i;
                break;
            }
        }
    }
}

```

```

        }
    }

    if (found != -1)
    {
        frequency[found]++;
    }
    else if (wordCount < MAX_WORDS)
    {
        strcpy(words[wordCount], word);
        frequency[wordCount] = 1;
        wordCount++;
    }

    word = strtok(NULL, " \t\n.,!?:\"'()[\]{}");
}

if (wordCount == 0)
{
    printf("No words found.\n");
    return;
}

printf("Word frequencies:\n");

for (j = 0; j < wordCount; j++)
{
    printf("%-20s : %d\n", words[j], frequency[j]);
}
}

```

3. Test-Case Table

Test	Input/Condition	Operation	Expected Result
1	C programming is interesting	Longest/shortest	Longest = programming, shortest = C
2	Hello world	Remove	Hello world
3	c PROGRAMMING is FUN	Title case	C Programming Is Fun
4	C is fun c is easy	Word	c=2, is=2, fun=1, easy=1
5	Hello HELLO hello	Word	hello=3
6	Multiple tabs/spaces	Remove spaces	Consecutive whitespace replaced by one space

7	Choice 8	Menu	Invalid choice!
8	Choice 5	Exit	Program terminates

4. Execution

```
[aaryanrajput@aaryans-macbook ~ % nano day41.c
[aaryanrajput@aaryans-macbook ~ % clang day41.c -o day41
[aaryanrajput@aaryans-macbook ~ % ./day41
Enter paragraph 1:
hello my name is aaryan i am in upes

Enter paragraph 2:
hello computer

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Exit
=====
Enter your choice: 1

--- Paragraph 1 ---
Longest word : aaryan
Shortest word : i

--- Paragraph 2 ---
Longest word : computer
Shortest word : hello

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Exit
=====
Enter your choice: 2

--- Paragraph 1 ---
Modified paragraph:
hello my name is aaryan i am in upes

--- Paragraph 2 ---
Modified paragraph:
hello computer

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Exit
=====
Enter your choice: 3

--- Paragraph 1 ---
Title case:
Hello My Name Is Aaryan I Am In Upes

--- Paragraph 2 ---
Title case:
Hello Computer
```

```
===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Exit
=====
Enter your choice: 4

--- Paragraph 1 ---
Word frequencies:
hello           : 1
my              : 1
name            : 1
is              : 1
aaryan          : 1
i               : 1
am              : 1
in              : 1
upes            : 1

--- Paragraph 2 ---
Word frequencies:
hello           : 1
computer        : 1

===== MENU =====
1. Find longest and shortest words
2. Remove repeated consecutive spaces
3. Convert paragraphs to title case
4. Find frequency of every distinct word
5. Exit
=====
Enter your choice: 5

Exiting program...
aaryanrajput@aaryans-macbook ~ %
```